

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

Figure 7: Ping traceability for host1 and host2

```
mininet> link s1 h1 down
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 100% dropped (0/2 received)
```

Figure 9: Bringing down the interface for host 1 connected to switch 1

This command will make each host in the network ping every other host in the network. In the network that we have, *h1* will ping *h2*, and *h2* will ping *h1*. As seen in Figure 7, the successful pings indicate that all the links in the network are active.

When we investigate the interface *eth1* of switch *s1* using Wireshark, we find that both the ping requests are successful (Figure 8).

The command to down a link is:

```
Mininet> link s1 h1 down
```

The above command will down the link between switch *s1* and host *h1* (Figure 9). Further, on pinging the hosts using the *pingall* command, we can see that both the pings are unsuccessful due to the link getting down.

- The command to build custom topology is:

```
#sudo mn --topo single,3
```

This command will create a topology as shown in Figure 12, and initialise the links and addresses of hosts and switches.

- The command to perform regression testing is:

```
#sudo mn --test pingpair
```

The above command is used to create a network with default topology, perform the *pingall* function and stop the network. This command is basically used to test how Mininet works.

- The command to open the xterm window is:

```
Mininet> h1 xterm
```

No.	Time	Source	Destination	Protocol	Length	Info
21	10.000126368	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request 1d=0x133c, seq=142/30832, ttl=64
22	10.000127293	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply 1d=0x133c, seq=142/30832, ttl=64
23	10.000127685	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request 1d=0x133c, seq=143/30860, ttl=64
24	10.000128046	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply 1d=0x133c, seq=143/30860, ttl=64
25	10.000128558	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request 1d=0x133c, seq=144/30884, ttl=64
26	10.000140589	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply 1d=0x133c, seq=144/30884, ttl=64

Figure 8: Ping packets (ICMP) captured

No.	Time	Source	Destination	Protocol	Length	Info
30	143.170775454	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request 1d=0x133c, seq=1/256, ttl=64 (no response)

Figure 10: Unsuccessful ping packets captured at interface *s1-eth1*

```
root@hp-HP-431-Notebook-PC:~# mn --topo single,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Figure 11: Topology created in Mininet

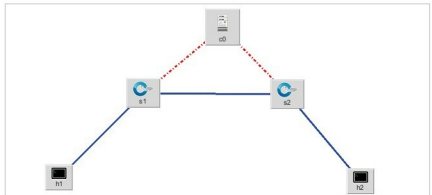


Figure 12: Topology creation using Minidit (an open source tool to create Mininet topologies)

```
root@hp-HP-431-Notebook-PC:~# mn --test pingpair
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
```

Figure 13a: Topology creation and validation tests at mining